Here's a comprehensive list of **40 Angular Interview Questions and Answers** tailored for candidates with **2–5+ years of experience**, covering **Angular 17+, RxJS, Signals, NgRx, and advanced concepts**:

🔹 Core Angular (Foundational & Advanced)

1. **What is Angular, and how is it different from AngularJS?**
   Answer: Angular (2+) is a complete rewrite of AngularJS, built in TypeScript. It uses a component-based architecture, supports RxJS for reactive programming, and is more performant due to Ahead-of-Time (AOT) compilation.
2. **What are Angular Modules?**
   Answer: Angular Modules (`@NgModule`) group components, directives, pipes, and services. They help organize code, allow lazy loading, and provide scope.
3. **What is the difference between `declarations` , `imports` , `exports` , and `providers` in `@NgModule` ?**
   Answer:
   - `declarations` : components, directives, and pipes that belong to the module.
   - `imports` : other modules whose exported classes are needed.
   - `exports` : declarations made available to other modules.
   - `providers` : services used within the module.
4. **What are Angular Components?**
   Answer: Components control a part of the UI. Each component has a class, HTML template, and CSS styles.
5. **What are Directives in Angular?**
   Answer:
   - **Structural** (`*ngIf` , `*ngFor`) – change DOM layout.
   - **Attribute** (e.g., `ngClass`) – change appearance or behavior of DOM elements.
6. **What is Change Detection in Angular?**
   Answer: Mechanism by which Angular checks for changes in data and updates the DOM. Angular uses zone.js to detect asynchronous operations and trigger change detection.
7. **Explain OnPush Change Detection Strategy.**
   Answer: It only checks components when @Input changes or when an event is triggered from inside. It improves performance.
8. **What are Angular Lifecycle Hooks?**
   Answer: Hooks like `ngOnInit()` , `ngOnChanges()` , `ngOnDestroy()` allow logic to run at specific points in component lifecycle.
9. **What is the purpose of `ngOnDestroy()` ?**
   Answer: Used to clean up resources like subscriptions or event listeners when a component is destroyed.
10. **What is ViewEncapsulation in Angular?**
    Answer: Controls CSS scope:

- `Emulated` (default): scopes styles to component.
- `None`: global styles.
- `ShadowDom`: uses native Shadow DOM.

🔹 Angular 17+ and Signals

11. **What are Signals in Angular?**
    Answer: A reactive primitive in Angular 17+ used to track and respond to state changes without zone.js. It replaces parts of RxJS in component logic.
12. **How do Signals differ from Observables?**
    Answer:
    - **Signals** are synchronous and pull-based.
    - **Observables** are push-based and asynchronous.
13. **What is `computed()` in Signals?**
    Answer: Creates derived signals based on other signals. It auto-recomputes when dependencies change.
14. **What is `effect()` in Signals?**
    Answer: Executes side effects when signal values change (e.g., updating DOM or making API calls).
15. **Can you combine Signals with RxJS?**
    Answer: Yes. `toObservable()` and `toSignal()` are utility functions for interoperability.

🔹 Routing & Lazy Loading

16. **What is Angular Routing?**
    Answer: Enables navigation among views/components using routes (`RouterModule`).
17. **What is Lazy Loading?**
    Answer: Loading feature modules only when required using `loadChildren`, improving initial load time.
18. **What is a Route Guard? Types?**
    Answer: Guards restrict navigation. Types:
    - `CanActivate`
    - `CanActivateChild`
    - `CanDeactivate`
    - `Resolve`
19. **What is a Resolver in Angular?**
    Answer: Pre-fetches data before navigating to a route, using `Resolve<T>` interface.

🔹 RxJS & Reactive Programming

20. **What is RxJS in Angular?**
    Answer: RxJS provides Observables for reactive programming in Angular. It powers forms, HTTP, and event handling.
21. **Difference between `Observable`, `Subject`, `BehaviorSubject`, and `ReplaySubject`?**
    Answer:
    - `Observable`: unicast stream.
    - `Subject`: multicast stream.
    - `BehaviorSubject`: emits current value to new subscribers.
    - `ReplaySubject`: replays past values to new subscribers.
22. **What are commonly used RxJS operators?**
    Answer: `map`, `switchMap`, `mergeMap`, `debounceTime`, `take`, `filter`, `distinctUntilChanged`.
23. **What is the difference between `switchMap` and `mergeMap`?**
    Answer:
    - `switchMap`: cancels previous inner observable.
    - `mergeMap`: flattens all inner observables concurrently.

🔹 Forms & Validation

24. **Difference between Template-driven and Reactive Forms?**
    Answer:
    - **Template-driven**: defined in HTML, suitable for simple forms.
    - **Reactive**: defined in TS, offers better control and validation.
25. **How do you add custom form validation?**
    Answer: Create a function that returns a `ValidationErrors` object or `null`, and register with form control.
26. **How do you disable a form control in Reactive Forms?**
    Answer: Use `control.disable()` or set it in the constructor with `{ disabled: true }`.

🔹 NgRx (State Management)

27. **What is NgRx?**
    Answer: A reactive state management library using Redux pattern and RxJS.
28. **NgRx Core Concepts?**
    Answer:

- **Store**: application state.
- **Actions**: describe events.
- **Reducers**: handle state changes.
- **Effects**: handle side effects (e.g., HTTP).

29. **What is an Effect in NgRx?**
    Answer: An injectable service that listens to actions and dispatches new ones after side effects.
30. **What is the role of Selectors in NgRx?**
    Answer: Pure functions used to select, project, and memoize parts of the state.

🔹 Performance & Optimization

31. **How to optimize Angular performance?**
    Answer:
    - Use `OnPush` strategy
    - Lazy load modules
    - Use `trackBy` in `ngFor`
    - Avoid memory leaks
    - Use `Pure Pipes`
32. **What are Pure vs Impure Pipes?**
    Answer:
    - **Pure**: executes only on input change (default).
    - **Impure**: executes every change detection cycle (can be slow).
33. **How to handle memory leaks in Angular?**
    Answer: Unsubscribe from Observables (e.g., use `takeUntil`, `async`, or `ngOnDestroy`).

🔹 Testing in Angular

34. **How do you test Angular components?**
    Answer: Use Jasmine/Karma. Key functions: `TestBed`, `fixture`, `componentInstance`, `detectChanges()`.
35. **What is TestBed in Angular Testing?**
    Answer: A testing utility that configures and initializes the environment for unit testing components.
36. **How to mock a service in Angular tests?**
    Answer: Use `useClass`, `useValue`, or `useFactory` in TestBed `providers`.

🔹 Advanced Topics

37. **What is Dependency Injection (DI)?**
    Answer: Angular's mechanism to inject dependencies (services, tokens) at runtime using the `@Injectable()` decorator.
38. **What are Injection Tokens in Angular?**
    Answer: Custom DI tokens used for injecting non-class dependencies (e.g., configuration values).
39. **What is a Standalone Component in Angular?**
    Answer: Introduced in Angular 14+, it allows components without being part of any NgModule by setting `standalone: true`.
40. **What is Hydration in Angular SSR?**
    Answer: In Angular Universal, hydration reuses server-rendered DOM on the client to improve performance and avoid flickering.